

Centro Estadual de Educação Tecnológica Paula Souza
ETEC Prof. Camargo Aranha
TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

Rubens Gabriel Policeno
Nathan Henrique Peres Cota
Matheus Marchese Calabro
Leonardo Verderame
Lorenzo Salvatore Bizuli Chiantelli
Vinícius Angelus

MONITORED
Sistema de monitoramento e gestão de computadores
em laboratórios escolares

São Paulo
2026

Rubens Gabriel Policeno
Nathan Henrique Peres Cota
Matheus Marchese Calabro
Leonardo Verderame
Lorenzo Salvatore Bizuli Chiantelli
Vinícius Angelus

MONITOREDU
Sistema de monitoramento e gestão de computadores
em laboratórios escolares

Trabalho de Conclusão de Curso apresentado
ao curso Técnico em Desenvolvimento de
Sistemas da ETEC Prof. Camargo Aranha,
como requisito parcial para obtenção do título
de Técnico em Desenvolvimento de Sistemas.

Orientador: Luiz Lima
Coorientador: Ricardo Palhares
Coorientador: Davi Vilar

São Paulo
2026

RESUMO

Este Trabalho de Conclusão de Curso apresenta o desenvolvimento do MonitorEdu, um sistema de monitoramento e apoio à gestão de computadores em laboratórios escolares, com foco no uso por professores e responsáveis por escolas públicas. O objetivo é centralizar informações das estações e permitir a execução de ações administrativas autorizadas por meio de uma interface web. A pesquisa é aplicada e experimental, com desenvolvimento incremental, análise de soluções semelhantes e validação do protótipo em ambiente controlado. A arquitetura atual integra uma interface desenvolvida em React, um servidor implementado em Cloudflare Workers, com persistência em Cloudflare D1, e um agente local desenvolvido em Go. A comunicação ocorre por requisições HTTP periódicas, mediante polling, para envio de métricas e consulta de comandos. O sistema permite acompanhar o uso de processador, memória e disco, consultar processos em execução, bloquear e desbloquear monitor, teclado e mouse, encerrar processos e aplicar políticas de bloqueio de domínios por meio do Sistema de Nomes de Domínio (DNS). Conta também com painel administrativo para o gerenciamento de usuários e instituições. A versão para Windows está disponível, enquanto os pacotes .deb, para distribuições baseadas em Debian, e .rpm, para Fedora e sistemas compatíveis, encontram-se em fase de finalização. Nos testes iniciais do agente, realizados em um notebook Positivo Master N1110, observaram-se consumo aproximado de 14 MB de memória RAM e uso do processador inferior a 1%. Essas medições descrevem o consumo observado no ambiente avaliado. O projeto busca adequar as funcionalidades ao contexto escolar, com configuração simplificada e baixo consumo de recursos.

Palavras-chave: monitoramento de computadores; laboratório de informática; desenvolvimento de sistemas; Cloudflare Workers; Go.

ABSTRACT

This final project presents the development of MonitorEdu, a computer monitoring and management support system for school laboratories, intended for teachers and staff at public schools. Its objective is to centralize workstation information and enable authorized administrative actions through a web interface. The research is applied and experimental, with incremental development, analysis of similar solutions, and prototype validation in a controlled environment. The current architecture combines a React interface, a server implemented using Cloudflare Workers with Cloudflare D1 persistence, and a local agent developed in Go. Communication relies on periodic HTTP requests through polling to send metrics and retrieve commands. The system allows users to monitor processor, memory, and disk usage, view running processes, lock and unlock the monitor, keyboard, and mouse, terminate processes, and apply domain blocking policies through the Domain Name System (DNS). It also includes an administrative panel for managing users and institutions. The Windows version is available, while .deb packages for Debian-based distributions and .rpm packages for Fedora and compatible systems are being finalized. Initial agent tests conducted on a Positivo Master N1110 notebook showed approximately 14 MB of RAM usage and processor usage below 1%. These measurements describe resource consumption in the evaluated environment. The project aims to adapt its features to the school context, with simplified configuration and low resource consumption.

Keywords: computer monitoring; computer laboratory; systems development; Cloudflare Workers; Go.

SUMÁRIO

1 INTRODUÇÃO.....	6
1.1 Problema de pesquisa.....	7
1.2 Justificativa.....	7
1.3 Objetivos.....	8
1.3.1 Objetivo geral.....	8
1.3.2 Objetivos específicos.....	8
2 FUNDAMENTAÇÃO TEÓRICA.....	10
2.1 Monitoramento de computadores em ambientes educacionais.....	10
2.2 Arquitetura cliente servidor.....	10
2.3 Comunicação HTTP e polling.....	11
2.4 Monitoramento de recursos computacionais.....	11
2.5 Usabilidade e configuração simplificada.....	11
2.6 Tecnologias utilizadas.....	12
2.6.1 React.....	12
2.6.2 Cloudflare Workers.....	12
2.6.3 Cloudflare D1.....	12
2.6.4 Go.....	13
2.6.5 Tecnologias utilizadas nos protótipos anteriores.....	13
2.7 Sistema de Nomes de Domínio e controle de acesso.....	13
3 ANÁLISE DE SOLUÇÕES EXISTENTES E DIFERENCIAIS DO MONITORED.....	14
3.1 Softwares de referência.....	14
3.2 Comparação dos requisitos técnicos publicados.....	14
3.3 Diferenciais do MonitorEdu no contexto proposto.....	16
4 METODOLOGIA.....	17
4.1 Caracterização da pesquisa.....	17
4.2 Etapas de desenvolvimento.....	17
4.3 Organização do desenvolvimento e versionamento.....	18
4.4 Ambiente de avaliação.....	18
4.5 Estratégia de validação.....	19
5 LEVANTAMENTO E ESPECIFICAÇÃO DE REQUISITOS.....	20
5.1 Público alvo e atores do sistema.....	20
5.2 Requisitos funcionais.....	20
5.3 Requisitos não funcionais.....	21
5.4 Regras de negócio.....	21
5.5 Delimitação do escopo atual.....	22
6 PROJETO E ARQUITETURA DA SOLUÇÃO.....	23
6.1 Visão geral da arquitetura.....	23
6.2 Interface web.....	23
6.3 Servidor central.....	24
6.4 Agente local.....	24
6.5 Fluxo de métricas e comandos.....	24
6.6 Persistência de dados.....	25
6.7 Fluxo de bloqueio por DNS.....	26
6.8 Instalação e configuração do agente.....	26
6.9 Estrutura do repositório.....	27
7 IMPLEMENTAÇÃO.....	28

7.1 Implementação da interface web.....	28
7.1.1 Painel do professor.....	28
7.1.2 Painel administrativo.....	28
7.2 Implementação da API em Cloudflare Workers.....	29
7.3 Implementação do agente local.....	30
7.3.1 Coleta de métricas e processos.....	30
7.3.2 Registro e execução de comandos.....	30
7.3.3 Serviço e sessão do usuário no Windows.....	31
7.3.4 Serviço e sessão do usuário no Linux.....	32
7.3.5 Bloqueio e desbloqueio do monitor.....	34
7.3.6 Configuração inicial e registro da estação.....	34
7.3.7 Distribuição para Windows e Linux.....	35
7.3.8 Registro e rotação de logs.....	35
7.4 Integração entre os componentes.....	36
7.5 Estado atual da implementação.....	36
8 TESTES E VALIDAÇÃO.....	37
8.1 Plano de testes.....	37
8.2 Cenários e critérios de verificação.....	37
8.3 Testes de integração e tratamento de falhas.....	38
8.4 Consumo observado do agente.....	38
8.5 Evidências e alcance da validação.....	39
9 RESULTADOS E DISCUSSÃO.....	40
9.1 Resultados obtidos.....	40
9.2 Relação com os objetivos definidos.....	40
9.3 Interpretação do comparativo de consumo.....	41
9.4 Limitações identificadas.....	42
10 CONSIDERAÇÕES FINAIS.....	43
10.1 Síntese do trabalho.....	43
10.2 Trabalhos futuros.....	43
REFERÊNCIAS.....	45

1 INTRODUÇÃO

A utilização de computadores em laboratórios escolares permite realizar pesquisas, produzir conteúdo e desenvolver atividades digitais. O uso simultâneo de diversos equipamentos também exige acompanhamento: o professor precisa observar o funcionamento das máquinas, identificar situações que se afastem dos objetivos da aula e executar ações administrativas quando necessário. A organização dessas tarefas compõe o problema de gestão abordado neste trabalho.

Em um laboratório com várias estações, a verificação presencial de cada computador pode aumentar o tempo dedicado a tarefas operacionais. Quando o professor ou o responsável pelo espaço também assume a administração dos equipamentos, uma ferramenta que concentre informações e comandos pode apoiar esse acompanhamento. O projeto considera esse cenário, com atenção à utilização em escolas públicas e à necessidade de uma configuração de fácil compreensão.

O MonitorEdu foi desenvolvido como um sistema de monitoramento e apoio à gestão de computadores em laboratórios escolares. A solução reúne informações das estações em uma interface web e permite o envio de comandos autorizados. Seu escopo prioriza funcionalidades relacionadas ao acompanhamento das máquinas e à administração do laboratório, em um projeto acadêmico do curso Técnico em Desenvolvimento de Sistemas.

A arquitetura atual integra uma interface desenvolvida em React, um servidor implementado em Cloudflare Workers, com persistência em Cloudflare D1, e um agente local desenvolvido em Go. Os protótipos anteriores utilizaram Express no servidor. A comunicação entre o agente e o servidor utiliza o Protocolo de Transferência de Hipertexto (HTTP), com consultas periódicas, denominadas polling, para envio de métricas e recebimento de comandos.

O sistema já dispõe de painel administrativo para usuários e instituições e de bloqueio de domínios por meio do Sistema de Nomes de Domínio (DNS). Também permite consultar métricas e processos e executar comandos de bloqueio e desbloqueio de monitor, teclado e mouse e de encerramento de processos. A versão para Windows está disponível. Para Linux, os pacotes .deb, destinados às distribuições baseadas em Debian, e .rpm, destinados ao Fedora e a sistemas compatíveis, encontram-se em fase de finalização.

O trabalho adota uma abordagem aplicada e experimental, com desenvolvimento incremental, análise de soluções semelhantes e testes em ambiente controlado. A validação inicial contempla o funcionamento dos componentes e o consumo de recursos do agente.

1.1 Problema de pesquisa

O problema de pesquisa consiste em acompanhar e administrar diferentes computadores de um laboratório escolar por meio de uma ferramenta centralizada, de fácil compreensão e com instalação compatível com o ambiente utilizado pela instituição.

A questão que orienta o trabalho é: como desenvolver um sistema para laboratórios escolares que permita centralizar informações dos computadores, executar ações administrativas autorizadas e manter uma configuração simples para o uso cotidiano por professores e responsáveis pelo laboratório?

1.2 Justificativa

A análise de Veyon, NetSupport School e LanSchool, utilizados como referências de gerenciamento de salas, e de AnyDesk, voltado ao acesso remoto, auxilia a delimitação do escopo do MonitorEdu. O projeto prioriza uma interface centralizada, configuração simplificada e funcionalidades compatíveis com as necessidades de acompanhamento dos equipamentos. A comparação entre as soluções orienta as escolhas de desenvolvimento, considerando seus diferentes modelos de implantação e requisitos técnicos.

O desenvolvimento do MonitorEdu integra conhecimentos de interface web, servidor, banco de dados, redes, comunicação entre processos, coleta de informações do sistema operacional, instalação de software e controle de versões. A articulação desses componentes permite aplicar os conteúdos do curso à construção de uma aplicação distribuída e avaliar sua integração por meio de testes.

O consumo de recursos é relevante para a utilização do agente em equipamentos escolares. Nos testes iniciais, realizados em um notebook Positivo Master N1110 com processador Intel Celeron N4020, 4 GB de memória RAM, 128 GB de armazenamento e Windows 11 Pro, foram observados aproximadamente 14 MB de uso de memória pelo agente e uso de CPU inferior a 1%. Essas medições caracterizam o ambiente avaliado e orientam a análise de viabilidade; não correspondem, por si só, à definição de requisitos mínimos absolutos.

1.3 Objetivos

1.3.1 Objetivo geral

Desenvolver e validar um sistema de monitoramento e apoio à gestão de computadores em laboratórios escolares, com interface web centralizada, painel administrativo, agente local nas estações e comunicação intermediada por um servidor em Cloudflare Workers.

1.3.2 Objetivos específicos

- Disponibilizar uma interface web para visualizar os computadores associados a uma sala e acessar as ações do sistema.
- Disponibilizar um painel administrativo para o gerenciamento de usuários e instituições.
- Implementar um servidor em Cloudflare Workers para receber informações dos agentes, organizar os dados e encaminhar comandos.
- Desenvolver um agente local em Go para coletar métricas do computador e enviá-las periodicamente ao servidor.
- Permitir a consulta de processos e a execução de comandos autorizados nas estações monitoradas.
- Aplicar políticas de bloqueio de domínios por DNS nos computadores monitorados.
- Utilizar comunicação HTTP com polling entre agente e servidor.

- Oferecer configuração inicial acessível, reduzindo a necessidade de edição manual de arquivos.
- Organizar o projeto em módulos que permitam manutenção, testes e evolução das funcionalidades.
- Manter a distribuição para Windows e finalizar os pacotes .deb e .rpm destinados às distribuições Linux compatíveis.
- Avaliar o funcionamento do sistema e o consumo de recursos do agente em ambiente controlado, registrando resultados e limitações.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Monitoramento de computadores em ambientes educacionais

O monitoramento de computadores em um laboratório reúne informações sobre as estações para apoiar sua administração. O responsável pode identificar os equipamentos associados a uma sala, verificar a utilização de recursos e solicitar ações nos computadores sob sua responsabilidade. No MonitorEdu, o acompanhamento se concentra em métricas e processos; a transmissão contínua da tela dos alunos não integra o escopo atual.

Soluções como o Veyon demonstram uma aplicação desse modelo ao contexto educacional, com componentes destinados ao computador do professor e às estações monitoradas. Sua documentação também trata da autenticação, da configuração e das condições de rede necessárias à operação (Veyon, 2026). Esses aspectos orientam a análise de soluções existentes, sem pressupor que todos os produtos possuam as mesmas funcionalidades ou necessidades de infraestrutura.

2.2 Arquitetura cliente servidor

A organização de uma aplicação em clientes e servidor distribui responsabilidades entre quem solicita serviços e quem processa as requisições. Na comunicação HTTP, um cliente envia uma requisição e o servidor produz uma resposta, conforme a semântica do protocolo (Fielding; Nottingham; Reschke, 2022). Essa separação permite que uma mesma API atenda clientes diferentes, desde que respeitem o formato das mensagens e as regras de acesso.

No MonitorEdu, a interface web e o agente são clientes da API. A interface apresenta as informações ao professor e encaminha solicitações de ações; o agente obtém dados do sistema operacional e executa ações locais. O servidor central relaciona usuários, salas e estações e mantém os dados necessários. A interface não precisa estabelecer uma conexão direta com cada computador monitorado.

2.3 Comunicação HTTP e polling

Polling é a consulta periódica de um serviço para obter atualizações. No projeto, o agente inicia requisições HTTP para enviar métricas e receber um comando pendente na resposta. Há também uma requisição específica para sincronizar a política DNS. O uso de HTTP não obriga a abrir uma nova conexão de transporte em cada consulta, pois a reutilização de conexões depende do cliente e do servidor; o que caracteriza o polling é a repetição das requisições.

O intervalo das consultas interfere no tempo de atualização. Uma ação registrada logo após o envio das métricas pode aguardar o próximo ciclo do agente. Além desse intervalo, devem ser considerados o tempo de rede, o processamento da API e a execução local. Reduzir o intervalo aumenta a frequência de requisições, de modo que sua escolha deve considerar a resposta esperada e a carga produzida. O comportamento adotado e seus limites são descritos no projeto da solução.

2.4 Monitoramento de recursos computacionais

As métricas coletadas descrevem aspectos diferentes do computador. A utilização de CPU indica o uso do processador no intervalo de observação; a memória pode ser expressa por percentual e quantidade utilizada; e os dados de disco informam ocupação e capacidade do volume consultado. A lista de processos relaciona aplicações em execução a seus identificadores, chamados PIDs. Esses indicadores devem ser interpretados junto ao instante da coleta e à finalidade de cada medida.

No agente, a biblioteca gopsutil fornece as consultas ao sistema operacional. O módulo collector organiza CPU, memória, disco e processos antes do envio. Uma rotina separada mede o próprio processo do agente. Assim, o uso total de memória da estação, apresentado ao professor, não deve ser confundido com a memória consumida pelo MonitorEdu, utilizada na avaliação de desempenho do agente.

2.5 Usabilidade e configuração simplificada

A interface precisa tornar compreensíveis as ações e seus resultados. Visibilidade do estado do sistema, consistência e prevenção de erros são princípios

descritos por Nielsen (1994) para o projeto de interfaces. No MonitorEdu, esses princípios orientam a organização das salas, a identificação do computador selecionado, a apresentação de mensagens de erro e a confirmação de ações que alteram o ambiente.

A configuração inicial utiliza o código de ingresso da sala, evitando que o usuário precise informar manualmente a identificação interna do agente. O identificador é gerado pelo programa e associado à sala durante o registro. Essa escolha reduz o número de informações solicitadas na instalação, embora não substitua a necessidade de permissões administrativas para os componentes que acessam serviços e configurações do sistema operacional.

2.6 Tecnologias utilizadas

2.6.1 React

React organiza a interface em componentes com lógica e apresentação próprias, permitindo combinar elementos menores em telas completas (React, 2026). No MonitorEdu, essa organização aparece em componentes para salas, computadores, processos, controles e administração. Os dados retornados pela API atualizam o estado da interface e determinam o conteúdo apresentado ao usuário.

2.6.2 Cloudflare Workers

Cloudflare Workers é a plataforma utilizada para executar o servidor atual. Ela permite publicar aplicações e APIs sem manter uma máquina de servidor dedicada sob administração direta do projeto (Cloudflare, 2026b). A implementação do MonitorEdu utiliza TypeScript e Hono para organizar rotas HTTP. Node.js e Express pertencem aos protótipos anteriores, preservados no repositório como parte da evolução do desenvolvimento.

2.6.3 Cloudflare D1

Cloudflare D1 é um serviço gerenciado de banco de dados com semântica SQL compatível com SQLite e acesso a partir de Workers (Cloudflare, 2026a). No sistema

atual, a persistência é acessada pelo Worker e mantém usuários, instituições, salas, agentes, métricas, comandos e políticas DNS. Essa arquitetura difere do banco SQLite local usado nos protótipos com Express, embora conserve a utilização de SQL.

2.6.4 Go

Go é a linguagem utilizada no agente local. Sua documentação apresenta recursos para desenvolvimento de programas compilados, bibliotecas de rede e mecanismos de concorrência (Go, 2026). No projeto, esses recursos sustentam a coleta periódica, a comunicação HTTP, o serviço DNS e a troca de mensagens entre o processo principal e a sessão do usuário. Arquivos específicos por sistema operacional implementam os comportamentos de Windows e Linux.

2.6.5 Tecnologias utilizadas nos protótipos anteriores

Os primeiros protótipos utilizaram Express, um framework para aplicações web em Node.js (Express, 2026), e SQLite, um mecanismo de banco de dados que pode operar no próprio processo da aplicação (SQLite, 2026). A permanência desses diretórios no repositório documenta a evolução do projeto. Na descrição do estado atual, o servidor corresponde ao Cloudflare Workers e a persistência ao Cloudflare D1.

2.7 Sistema de Nomes de Domínio e controle de acesso

O Sistema de Nomes de Domínio, ou DNS, organiza a resolução de nomes na rede, permitindo que aplicações consultem informações associadas a um domínio (Mockapetris, 1987). Um filtro DNS pode decidir se uma consulta será encaminhada ao resolvedor externo ou receberá uma resposta de bloqueio. Esse controle atua sobre a resolução do nome e não equivale à análise do conteúdo de cada página acessada.

O MonitorEdu aplica listas por estação: no modo de bloqueio, recusa os domínios listados; no modo de permissão, admite apenas os domínios listados e seus subdomínios. O controle depende do uso do resolvedor local do agente e não abrange automaticamente consultas feitas por outros mecanismos.

3 ANÁLISE DE SOLUÇÕES EXISTENTES E DIFERENCIAIS DO MONITOREDUC

3.1 Softwares de referência

A análise inclui Veyon, NetSupport School e LanSchool Air, voltados à administração de computadores em ambientes de ensino, e AnyDesk, voltado ao acesso e suporte remoto. A comparação considera informações divulgadas pelos próprios fornecedores. Seu objetivo é contextualizar as condições de implantação e o escopo do MonitorEdu, sem atribuir a produtos com finalidades distintas resultados obtidos no mesmo ensaio.

3.2 Comparação dos requisitos técnicos publicados

A Tabela 1 apresenta sistemas compatíveis, requisitos divulgados e recomendações de implantação. N/I indica que a documentação consultada não informa um valor mínimo numérico para o item. Essa indicação não significa consumo nulo. Quando uma informação corresponde a recomendação de rede, consumo por estação monitorada ou modalidade de instalação, o contexto é mantido na própria célula.

Tabela 1 - Requisitos e recomendações publicados para os softwares analisados

REQUISITO	SOFTWARES ANALISADOS			
	Veyon	AnyDesk	NetSupport School	LanSchool Air
Sistemas operacionais	Windows 10/11; Linux: Debian 10+, Ubuntu 20.04+, openSUSE 15.4+, Fedora 38+ e CentOS 7.9+.	Windows XP SP2 ou posterior; macOS 11+; Ubuntu 18.04+, Debian 10+, RHEL 8+, openSUSE e Linux Mint. Recursos variam por sistema.	Windows 7, 8/8.1, 10 e 11; outros ambientes conforme o componente.	Aluno: Windows 10 v1903+ ou 11 v21H2+, ChromeOS estável/LTS e macOS compatível. Professor: navegador web.
Memória RAM	Pelo menos 2 GB no Veyon Master; uso adicional de 20–30 MB no Master por computador monitorado, dependendo da resolução.	N/I	N/I	N/I
Processador e vídeo	Sistema com 2–4 núcleos de CPU recomendado.	Aceleração gráfica por hardware indicada para renderização e resposta.	N/I	Windows: x86-64 e ARM suportados. CPU mínima numérica: N/I.
Espaço livre em disco	N/I	N/I	250 MB: Aluno; 400 MB: Tutor; 400 MB: Técnico de TI; 600 MB: instalação completa.	N/I
Rede e conexão	TCP/IP. Rede Gigabit recomendada para mais de 10 computadores; Wi-Fi IEEE 802.11n ou superior.	TCP 80, 443 ou 6568: pelo menos uma disponível. UDP 50001–50003: descoberta local.	Conectividade de rede; regras de firewall conforme instalação. Velocidade mínima: N/I.	Monitoramento de telas: pico de 16–32 kbps por aluno; recomendação de 0,5 Mbps ao professor para 30 estações. TCP 80/443.
Instalação e permissões	Instalador Windows executado como administrador.	Modo portátil ou instalado; recursos dependem do modo e das permissões.	Componentes de professor e aluno conforme a finalidade da instalação.	Agente e extensão no aluno; instalação Windows requer administrador local.
Interface de administração	Aplicação Veyon Master.	Aplicação de acesso remoto.	Aplicação Tutor; resolução de 1024 × 768 ou superior.	Interface web; Chrome, Edge ou Safari atuais recomendados.

Fonte: elaborado pelos autores com base em Veyon (2026), AnyDesk (2025, 2026a, 2026b), NetSupport School (2026) e LanSchool Air (2026a, 2026b).

Nota: N/I = valor mínimo não informado na documentação consultada. Requisitos e recomendações não constituem medições feitas pelo grupo.

3.3 Diferenciais do MonitorEdu no contexto proposto

O MonitorEdu concentra o acompanhamento em CPU, memória, disco, processos e comandos administrativos definidos. A interface web permite ao professor acessar as salas sem instalar um aplicativo próprio de administração em seu computador. O agente instalado nas estações reúne as funções locais, enquanto o servidor atual opera em Cloudflare Workers e mantém a persistência em D1.

A configuração por código de sala e a separação entre usuários comuns e administradores atendem à organização proposta para o laboratório. O painel administrativo já permite cadastrar instituições e usuários, e o bloqueio DNS já integra a versão atual. A distribuição Windows está disponível; as distribuições Linux em .deb e .rpm permanecem em finalização.

As medições iniciais de aproximadamente 14 MB de RAM e CPU inferior a 1% indicam baixo consumo do agente no notebook avaliado. Contudo, esses dados não permitem concluir superioridade de desempenho em relação aos produtos analisados. O consumo adicional do Veyon Master por estação e os requisitos de espaço livre do NetSupport School descrevem componentes e condições diferentes. Uma comparação de eficiência exigiria executar funções equivalentes, no mesmo equipamento e com o mesmo protocolo de observação.

4 METODOLOGIA

4.1 Caracterização da pesquisa

O trabalho caracteriza-se como pesquisa aplicada ao desenvolvimento de um sistema para laboratórios escolares. O componente experimental consiste na implementação do protótipo e na observação de seu funcionamento em ambiente controlado. A análise combina a consulta a documentação técnica, a inspeção do código-fonte, a comparação de soluções existentes e registros iniciais de utilização e consumo do agente.

A investigação se concentra na viabilidade técnica da solução. Não foram produzidos dados que permitam avaliar impacto pedagógico, ganhos de aprendizagem ou satisfação de uma população de professores e alunos. Essas questões exigem um estudo de campo específico e não são inferidas a partir da disponibilidade das funcionalidades.

4.2 Etapas de desenvolvimento

O desenvolvimento foi organizado de forma incremental. A definição do problema e a análise de ferramentas semelhantes orientaram o levantamento dos requisitos. A arquitetura foi dividida em interface web, servidor central e agente local. Cada componente foi implementado por módulos, permitindo ajustar responsabilidades e integrar os fluxos de identificação, coleta de dados e execução de ações.

Entre a concepção inicial e a versão atual, o modelo de comunicação passou a utilizar HTTP com polling, e o agente foi desenvolvido em Go. Protótipos do servidor utilizaram Node.js, Express e SQLite local. Na arquitetura atual, o backend foi migrado para Cloudflare Workers, com persistência em D1. Essa evolução é registrada como mudança de implementação, sem apresentar os componentes antigos como dependências obrigatórias da versão atual.

Após a integração básica, o projeto incorporou o painel administrativo e o controle DNS por políticas de domínios. A distribuição Windows e a separação entre serviço e sessão do usuário fazem parte da implementação. Os scripts de

empacotamento Linux organizam os componentes em .deb e .rpm, mas a conclusão da distribuição depende da finalização e da validação desses pacotes nos ambientes correspondentes.

4.3 Organização do desenvolvimento e versionamento

O código-fonte é mantido no repositório oficial do projeto, com diretórios separados para interface, Worker e agente. O controle de versão registra mudanças nos módulos e permite relacionar a documentação ao estado da implementação. A análise técnica deste trabalho considera a revisão eef68ef do repositório, disponível em 30 de setembro de 2026 (3DBMonitoraEdu, 2026).

A inspeção considerou os módulos de comunicação, métricas, comandos, autenticação, administração e DNS, além dos arquivos de configuração e instalação para Windows e Linux. Os diretórios de protótipos com Express e os arquivos de execução local foram tratados como registros do desenvolvimento. A presença de um arquivo no repositório não foi usada, isoladamente, para declarar uma funcionalidade concluída ou um teste aprovado.

4.4 Ambiente de avaliação

Os testes iniciais de consumo do agente foram realizados em um notebook escolar Positivo Master N1110. A Tabela 2 registra as especificações informadas para o equipamento. Elas identificam o ambiente de observação e não estabelecem um requisito mínimo absoluto para o MonitorEdu.

Tabela 2 - Equipamento utilizado nos testes iniciais

Item	Especificação
Modelo	Positivo Master N1110
Processador	Intel Celeron N4020
Memória RAM	4 GB
Armazenamento	128 GB
Sistema operacional	Windows 11 Pro

Fonte: registros de teste do projeto MonitorEdu (2026).

4.5 Estratégia de validação

A validação relaciona cada requisito ao fluxo que o implementa e à evidência disponível. A leitura do código identifica rotinas e controles existentes; as capturas de tela registram estados da interface; e as medições iniciais descrevem o consumo observado. Esses tipos de evidência possuem alcances distintos: uma captura de tela não substitui uma série de medições, e a existência de uma rota não comprova todos os cenários de falha da integração.

Os cenários definidos abrangem registro de agente, atualização de métricas, entrega de comandos, bloqueio e desbloqueio, políticas DNS, acesso administrativo e comportamento diante de interrupções de rede. A comprovação completa exige registrar configuração, procedimento, resultado esperado e resultado observado em cada execução. Os ensaios sem registro de execução consolidado são tratados como parte do plano de validação, preservando a distinção entre implementação e resultado experimental.

5 LEVANTAMENTO E ESPECIFICAÇÃO DE REQUISITOS

5.1 Público alvo e atores do sistema

O público-alvo principal é formado por professores e responsáveis por laboratórios escolares. O professor utiliza a interface web para administrar suas salas, acompanhar as estações e solicitar ações. O administrador utiliza o ambiente restrito para cadastrar instituições e usuários. O agente representa o computador monitorado, enviando métricas e executando localmente os comandos previstos.

A sala relaciona as estações ao professor responsável. O código de ingresso permite registrar um agente nessa sala, e o identificador do agente diferencia cada computador nas mensagens trocadas com o servidor. A instituição compõe o cadastro administrativo dos usuários. Esses elementos possuem funções diferentes e não devem ser tratados como uma única identificação.

5.2 Requisitos funcionais

A Tabela 3 reúne os comportamentos previstos no escopo atual. As ações locais dependem das permissões e dos recursos do sistema operacional. A existência das funções no código não significa que a distribuição Linux, ainda em finalização, possua validação completa em todas as distribuições e sessões gráficas.

Tabela 3 - Requisitos funcionais do MonitorEdu

Código	Descrição
RF01	Autenticar o usuário e disponibilizar a interface correspondente ao seu perfil.
RF02	Criar, listar e remover salas sob responsabilidade do professor, conforme as regras de acesso.
RF03	Registrar o agente por código de sala e identificação própria da estação.
RF04	Listar os computadores associados à sala e apresentar seu último estado recebido.
RF05	Receber e apresentar CPU, memória, disco e processos das estações.
RF06	Registrar um comando destinado a um agente selecionado e entregá-lo no ciclo de métricas.
RF07	Bloquear e desbloquear monitor, teclado e mouse quando suportado pelo ambiente.
RF08	Solicitar o encerramento de um processo identificado por PID.
RF09	Configurar e sincronizar políticas DNS nos modos de bloqueio e permissão de domínios.
RF10	Registrar os domínios consultados pelo resolvedor local e enviá-los na sincronização DNS.
RF11	Disponibilizar painel restrito para cadastro e listagem de usuários e instituições.

Código	Descrição
RF12	Configurar o ingresso do agente por interface gráfica e salvar os dados de configuração.
RF13	Iniciar o serviço e o componente de sessão conforme a distribuição e o sistema operacional.

Fonte: elaborado pelos autores (2026).

5.3 Requisitos não funcionais

Os requisitos não funcionais orientam a qualidade e a operação da solução. As métricas de consumo servem como referência inicial para a avaliação de desempenho, sem substituir ensaios em equipamentos e cargas diferentes.

Tabela 4 - Requisitos não funcionais do MonitorEdu

Código	Categoria	Descrição
RNF01	Usabilidade	Apresentar salas, estações, ações e mensagens em linguagem direta e com identificação clara do alvo.
RNF02	Comunicação	Utilizar HTTP com polling e endereço HTTPS para a API atual, com intervalo configurável no agente.
RNF03	Desempenho	Manter consumo compatível com o uso escolar e avaliar CPU, RAM, armazenamento e logs separadamente.
RNF04	Modularidade	Separar interface, API, coleta, comandos, DNS e comunicação entre processos.
RNF05	Persistência	Manter os dados centrais no Cloudflare D1, com estrutura e alterações de esquema versionadas.
RNF06	Controle de acesso	Restringir as operações do professor às salas e agentes sob sua responsabilidade e a administração ao perfil autorizado.
RNF07	Execução local	Aceitar apenas ações registradas no executor, com parâmetros reconhecidos.
RNF08	Distribuição	Disponibilizar Windows e finalizar os pacotes .deb e .rpm com os componentes correspondentes.
RNF09	Rastreabilidade	Registrar eventos e erros; aplicar rotação com limite de 10 MB por arquivo de log ativo.

Fonte: elaborado pelos autores (2026).

5.4 Regras de negócio

As regras da Tabela 5 relacionam identificação, autorização e execução das ações. O cadastramento de uma estação e a autorização de uma ação pelo professor são etapas diferentes: o primeiro associa o computador à sala; a segunda verifica quem pode consultar ou alterar esse recurso.

Tabela 5 - Regras de negócio do MonitorEdu

Código	Regra
RN01	Cada agente possui UUID e pertence a uma sala registrada.
RN02	A sala possui professor responsável; consultas e ações sobre seus agentes verificam essa responsabilidade.
RN03	O ingresso do agente exige código de sala e registro aceito pelo servidor.
RN04	O agente aguarda identificação e código de sala antes de iniciar o ciclo normal de operação.
RN05	Cada comando é associado ao agente selecionado e interpretado por um conjunto definido de ações.
RN06	No modo blocklist, domínios listados são bloqueados; no modo allowlist, domínios não listados são bloqueados.
RN07	O cadastro administrativo de usuários associa a conta a uma instituição e ao perfil previsto.
RN08	As funções administrativas locais são utilizadas em computadores e ambientes autorizados pela instituição.

Fonte: elaborado pelos autores (2026).

5.5 Delimitação do escopo atual

O escopo atual inclui identificação das estações, métricas, processos, comandos locais, políticas DNS e painel administrativo. Não integra transmissão contínua de telas, chat, controle geral da área de trabalho ou relatórios históricos completos. O armazenamento de métricas mantém o estado mais recente de cada agente, adequado ao acompanhamento atual, e não uma série histórica de todas as coletas.

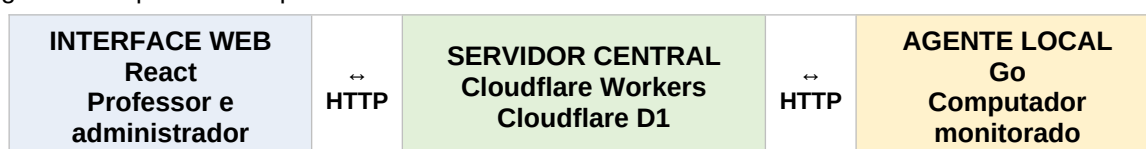
A versão Windows está disponível. A implementação Linux e os arquivos de empacotamento já existem, enquanto a distribuição em .deb e .rpm permanece em finalização. A compatibilidade das ações gráficas depende do ambiente utilizado; a presença de código para Linux não permite afirmar suporte universal a todas as distribuições, arquiteturas ou servidores gráficos.

6 PROJETO E ARQUITETURA DA SOLUÇÃO

6.1 Visão geral da arquitetura

O MonitorEdu reúne interface web, servidor central e agente local. O professor e o administrador utilizam a interface conforme suas permissões. A API em Cloudflare Workers recebe requisições, verifica regras de acesso e consulta o Cloudflare D1. O agente instalado nas estações coleta informações do computador, sincroniza políticas DNS e recebe comandos por requisições periódicas. A Figura 1 mantém a representação simplificada dessas responsabilidades.

Figura 1 - Arquitetura simplificada do MonitorEdu



Fonte: elaborado pelos autores com base na implementação do MonitorEdu (2026).

As setas representam comunicação com a API, sem indicar uma conexão direta entre navegador e estação. A persistência integra a camada do servidor; ela não é instalada em cada computador monitorado. A execução de ações que dependem da sessão gráfica ocorre no agente, conforme os mecanismos próprios de cada sistema operacional.

6.2 Interface web

A aplicação React organiza a navegação entre salas, computadores e detalhes da estação. O professor consulta as salas associadas à sua conta, seleciona uma estação e visualiza métricas e processos. Os controles geram solicitações à API, e a interface apresenta mensagens sobre o envio ou sobre erros de comunicação. O painel administrativo utiliza telas e operações específicas para usuários e instituições.

O painel não executa comandos diretamente no sistema operacional do aluno. Uma solicitação enviada pela interface é primeiro processada pelo servidor e, depois, entregue ao agente no ciclo de comunicação. A indicação de comando enviado confirma essa etapa de solicitação, mas não equivale, por si só, à comprovação de execução local concluída.

6.3 Servidor central

O backend atual está no diretório projeto/moniedu-worker. Hono organiza as rotas, e as funções de serviço executam consultas ao D1. A API oferece registro de agentes, recepção de métricas, sincronização DNS, salas e comandos. A autenticação de usuários é implementada com Better Auth, incluindo o componente administrativo utilizado para criar e listar contas.

As operações do professor sobre salas e agentes verificam a sessão e a responsabilidade pelo recurso. As rotas de instituições exigem perfil administrativo. O código também utiliza controles de origem e proteção nas requisições que alteram recursos. Esses mecanismos descrevem medidas implementadas, mas não constituem uma certificação de segurança nem comprovam que todos os possíveis cenários de acesso foram testados.

6.4 Agente local

O agente em Go executa as funções que exigem acesso à estação. O processo principal carrega a configuração, coleta métricas, comunica-se com a API e mantém o resolvidor DNS local. Um componente da sessão do usuário recebe comandos e participa da identificação dos processos associados às aplicações abertas. Essa separação atende à diferença entre o contexto de serviço e o contexto gráfico do usuário.

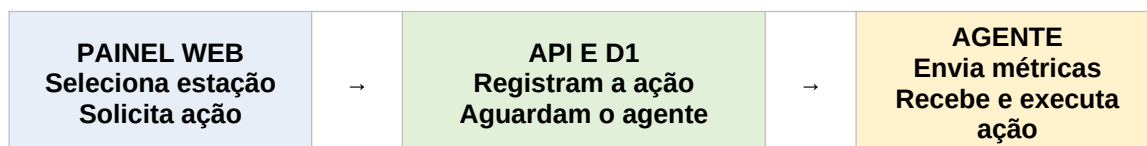
Os módulos collector, apiclient, executor, ipc, config, setup, dns e logger concentram responsabilidades diferentes. Arquivos específicos para Windows e Linux mantêm uma interface comum de funcionamento, mas utilizam mecanismos locais distintos. A organização permite localizar uma rotina sem presumir que a mesma chamada ao sistema operacional seja válida nas duas plataformas.

6.5 Fluxo de métricas e comandos

O agente realiza uma coleta inicial e repete o processo no intervalo configurado. As métricas são enviadas à API com o UUID da estação. O servidor atualiza o estado persistido e devolve um comando pendente, quando houver. O processo principal encaminha a ação ao componente de sessão, que interpreta o comando e executa a

rotina correspondente. A Figura 2 resume o percurso de uma solicitação feita pelo professor.

Figura 2 - Encaminhamento de comandos pelo ciclo de polling



Fonte: elaborado pelos autores com base na implementação do MonitorEdu (2026).

Na configuração padrão do agente, `interval_secs` vale 30 segundos. A listagem de estações no painel também é atualizada por uma consulta a cada 30 segundos. Esses intervalos pertencem a componentes diferentes: um determina quando a estação envia dados, e o outro determina quando a interface consulta o servidor. A lista de processos é solicitada ao selecionar a estação ou atualizar sua visualização. Não há evidência de atualização instantânea de todas essas informações.

No fluxo atual, o servidor remove o comando pendente quando o devolve na resposta das métricas. Não há, nesse percurso, confirmação posterior de que a sessão executou a ação com sucesso. O registro de comandos também utiliza substituição por agente, de modo que não deve ser descrito como uma fila ilimitada de ações. Esses pontos delimitam a confiabilidade que a versão atual permite afirmar e orientam melhorias futuras de confirmação e tratamento de falhas.

6.6 Persistência de dados

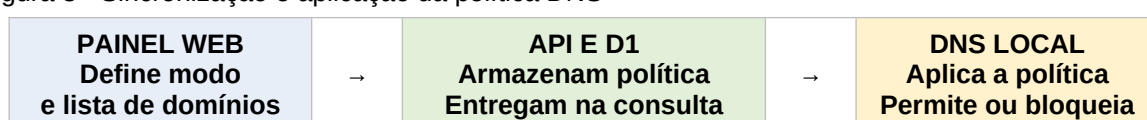
O Cloudflare D1 mantém os dados centrais da aplicação. Usuários e instituições compõem o cadastro administrativo; salas relacionam professores e estações; agentes mantêm a identificação de cada computador; e métricas, comandos e políticas DNS sustentam a operação. As migrações versionadas registram as alterações do esquema e permitem acompanhar a evolução da persistência.

A estrutura de métricas mantém um registro por agente e substitui seus valores a cada atualização. A lista de processos é armazenada em JSON, com seleção de até 50 registros, priorizados pelo instante de criação e, em seguida, pelo consumo de memória. Portanto, a lista apresentada não deve ser interpretada como inventário completo de todos os processos do sistema. As políticas DNS também são associadas ao agente e mantêm o modo e as listas utilizadas.

6.7 Fluxo de bloqueio por DNS

O bloqueio DNS já funciona na versão atual. O professor define uma política para a estação, e a API persiste o modo e as listas no D1. Durante a sincronização, o agente envia os domínios consultados pelo resolvidor local e recebe a política vigente. O modo recebido substitui a política em memória utilizada na decisão sobre consultas posteriores. A Figura 3 destaca essa relação entre definição, sincronização e aplicação local.

Figura 3 - Sincronização e aplicação da política DNS



Fonte: elaborado pelos autores com base na implementação do MonitorEdu (2026).

O resolvidor do agente atende consultas UDP em 127.0.0.1, porta 53. Consultas recusadas recebem resposta de nome inexistente, identificada como NXDOMAIN. Consultas admitidas são encaminhadas ao resolvidor externo 1.1.1.1, porta 53. O módulo também normaliza os nomes e considera subdomínios correspondentes aos domínios listados. A sincronização com a API e a consulta ao resolvidor externo são comunicações diferentes.

Em blocklist, a correspondência com a lista provoca o bloqueio; em allowlist, a ausência de correspondência provoca o bloqueio. O serviço permanece dependente da configuração DNS da estação e da disponibilidade de sua comunicação externa. O histórico enviado descreve consultas de nomes feitas pelo resolvidor local; não representa URLs completas, conteúdo das páginas ou prova de que um usuário visualizou cada domínio registrado.

6.8 Instalação e configuração do agente

A instalação distribui o processo principal e o componente de sessão. O serviço deve iniciar em segundo plano, enquanto o componente gráfico inicia no contexto do usuário. A configuração contém endereço do servidor, intervalo de coleta, volume consultado, código de sala e UUID. O registro no servidor associa esses dados à estação antes do funcionamento normal.

O Windows utiliza o instalador e as rotinas do serviço. No Linux, os pacotes .deb e .rpm organizam binários, configuração, unidade systemd e inicialização

da sessão gráfica. A distribuição Linux permanece em finalização, e a validação deve considerar as dependências utilizadas nas ações locais, como xrandr e xinput. O processo é detalhado no capítulo de implementação.

6.9 Estrutura do repositório

O Quadro 1 relaciona as responsabilidades descritas à organização do código. Os nomes dos diretórios são preservados como identificadores técnicos, mesmo quando conservam nomenclaturas anteriores do projeto.

Quadro 1 - Organização principal do código do MonitorEdu

Diretório ou arquivo	Responsabilidade
projeto/front	Aplicação React, páginas e componentes da interface.
projeto/moniedu-worker	API atual em Cloudflare Workers e configuração de implantação.
projeto/moniedu-worker/src/routes	Rotas para agentes, salas, comandos e instituições.
projeto/moniedu-worker/src/services	Consultas, regras de acesso, métricas e políticas DNS.
projeto/moniedu-worker/migrations	Alterações versionadas do esquema do D1.
projeto/agente/cmd	Entradas do agente principal e do componente de sessão.
projeto/agente/internal/collector	Coleta das métricas da estação e do processo do agente.
projeto/agente/internal/apiclient	Registro, envio de métricas e sincronização DNS.
projeto/agente/internal/executor	Ações locais registradas por sistema operacional.
projeto/agente/internal/ipc	Named Pipes no Windows e sockets Unix no Linux.
projeto/agente/internal/config e setup	Leitura, gravação e configuração inicial.
projeto/agente/internal/dns e logger	Resolvedor e políticas DNS; registro e rotação de logs.
projeto/agente/MonitorEduSetup.iss	Instalador Windows e integração com o serviço.
projeto/agente/build-deb.sh e build-rpm.sh	Compilação e empacotamento Linux em finalização.
projeto/servidor	Servidor Express dos protótipos anteriores.

Fonte: elaborado pelos autores com base no repositório oficial do MonitorEdu (2026).

7 IMPLEMENTAÇÃO

A implementação relaciona a arquitetura às rotinas presentes no código e às interfaces do sistema. As descrições distinguem a versão Windows disponível, os componentes Linux em finalização e os recursos já funcionais, como o painel administrativo e o bloqueio por DNS.

7.1 Implementação da interface web

7.1.1 Painel do professor

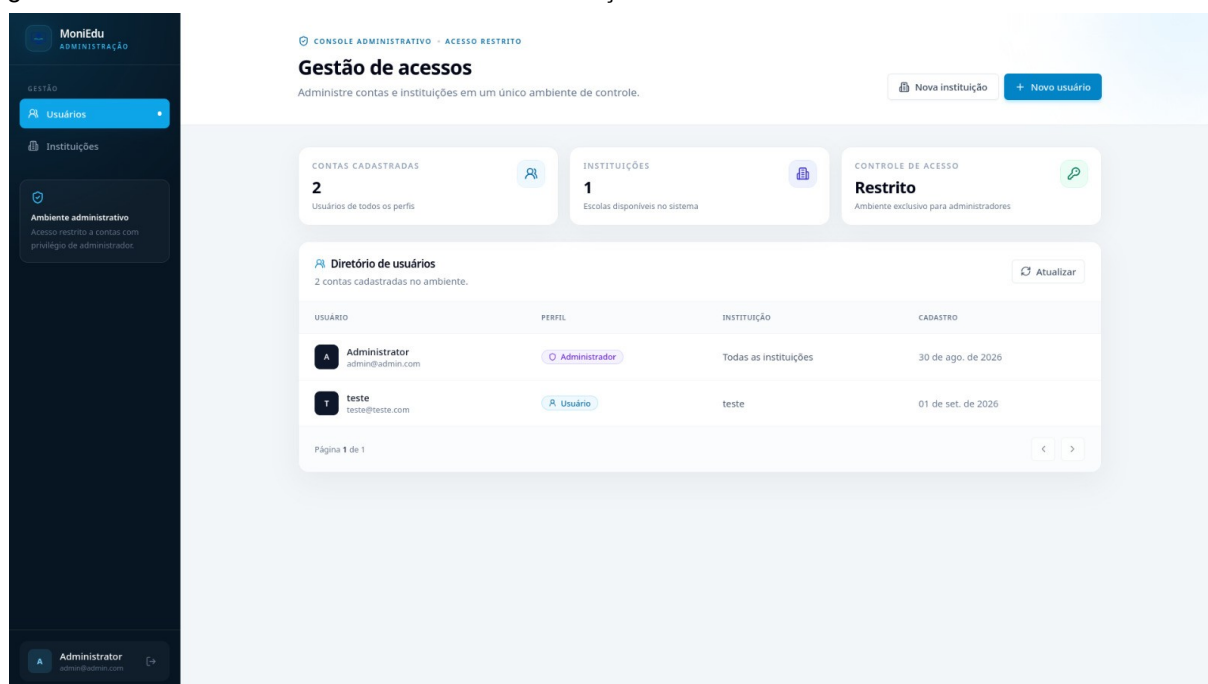
A página Dashboard carrega as salas da conta autenticada e permite criar ou remover uma sala. Ao entrar em uma sala, o professor acessa sua lista de estações. A seleção de um computador abre seus detalhes e processos, enquanto os controles encaminham ações ao servidor. Componentes separados organizam a navegação, a listagem de salas, os dados da estação e as mensagens de retorno.

A interface mantém estados para carregamento, erros e solicitações em andamento. O envio de um comando apresenta uma mensagem com a estação escolhida, ajudando o professor a reconhecer o alvo. A atualização periódica da lista de agentes utiliza o último estado disponível na API. A ausência de dados recentes precisa ser interpretada como condição de comunicação ou atualização, sem atribuir automaticamente uma causa à estação.

7.1.2 Painel administrativo

O painel administrativo já integra o sistema e apresenta usuários e instituições em um ambiente restrito. A implementação permite listar contas, consultar instituições, cadastrar uma instituição e criar um usuário associado a ela. O cadastro de usuário utiliza as operações administrativas do Better Auth, enquanto o cadastro de instituições utiliza a API do Worker. A Figura 4 registra a tela de gestão de acessos.

Figura 4 - Painel administrativo de usuários e instituições



Fonte: acervo do projeto MonitorEdu (2026).

Na captura, os totais de contas e instituições representam o estado daquele ambiente no momento do registro. Eles não constituem número de usuários participantes de uma pesquisa, quantidade de escolas atendidas ou resultado de teste de capacidade. A identificação MoniEdu ainda visível na tela pertence ao material gráfico do projeto; a nomenclatura adotada neste trabalho é MonitorEdu.

7.2 Implementação da API em Cloudflare Workers

A entrada da aplicação registra as rotas em Hono e aplica os controles correspondentes a cada grupo. O registro do agente recebe código de sala, UUID e nome do computador. A rota de métricas verifica os campos exigidos, a validade dos valores numéricos e a lista de processos antes de persistir a atualização. A resposta dessa rota pode conter uma ação pendente para a estação.

As funções de serviço utilizam consultas parametrizadas ao D1. Para a interface do professor, a autorização relaciona o agente à sala e verifica seu responsável. As rotas de política DNS permitem consultar e alterar listas após a verificação de acesso. A recepção das métricas e a sincronização feitas pelo agente utilizam sua identificação, o que exige avaliação específica de autenticação das estações antes de uma implantação ampliada.

A autenticação de usuários utiliza e-mail e senha, com cadastro público desabilitado e criação de contas pelo fluxo administrativo. A existência desses mecanismos não elimina a necessidade de revisar permissões, tratamento de entradas, proteção dos identificadores e registros de operação. A documentação diferencia os controles existentes das melhorias que ainda devem ser verificadas em testes de segurança.

7.3 Implementação do agente local

7.3.1 Coleta de métricas e processos

A coleta consulta CPU, memória e ocupação do volume configurado. Os resultados são organizados com nomes de campos definidos pela API e enviados como JSON. A memória da estação inclui quantidade utilizada e capacidade total; o disco inclui ocupação e capacidade do volume. O percentual de CPU da estação é obtido em uma observação da biblioteca gopsutil e não corresponde ao percentual do processo do agente.

O componente de sessão encaminha PIDs de aplicações ao processo principal por IPC. Quando esse relatório está disponível, o agente utiliza esses identificadores para compor a lista de processos enviada ao servidor, incluindo nome, uso de memória e instante de criação quando obtidos. O Worker valida e seleciona os registros antes de armazená-los. Essa seleção ajuda a delimitar o conteúdo apresentado, mas não constitui um inventário de todos os processos de todos os usuários.

7.3.2 Registro e execução de comandos

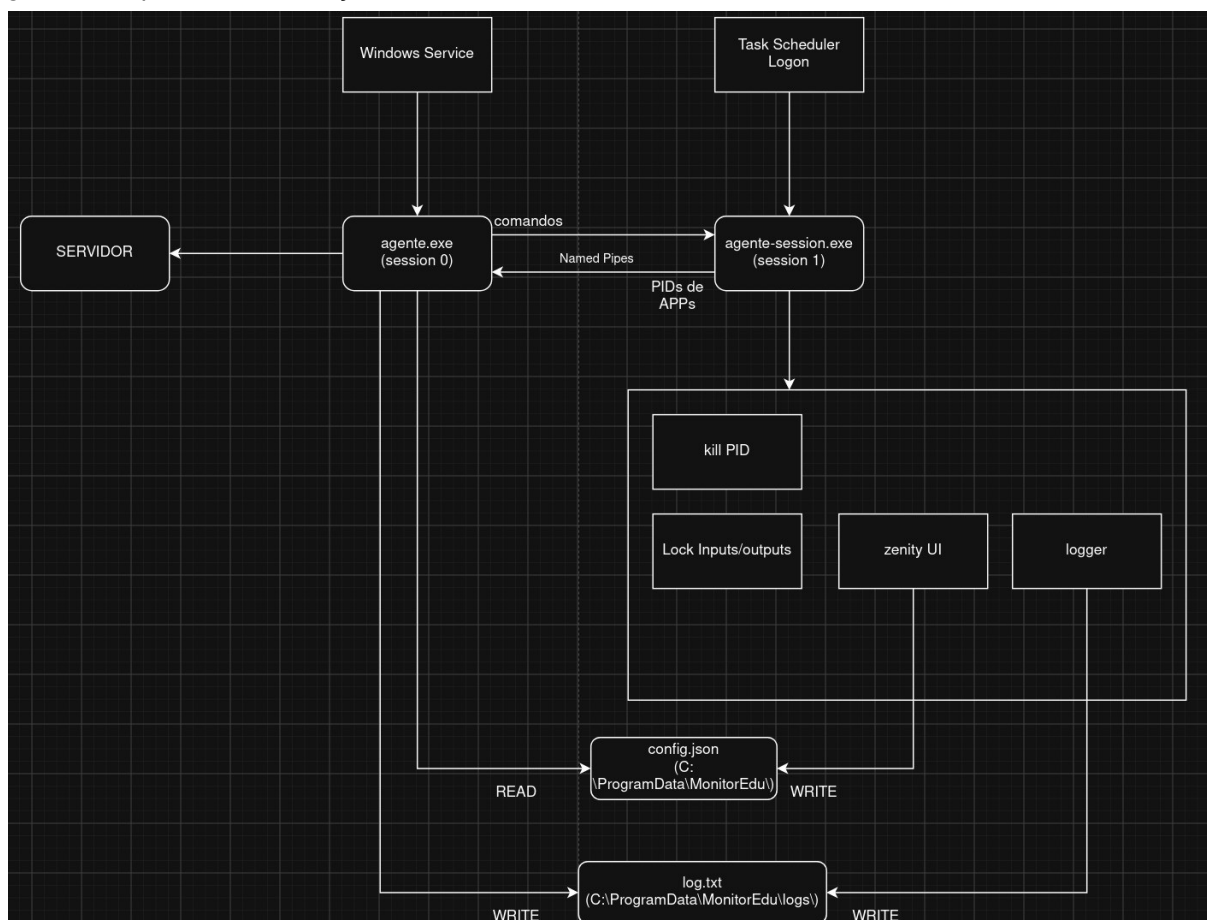
O executor mantém um conjunto explícito de ações: `lock_mouseAndKeyboard`, `unlock_mouseAndKeyboard`, `lock_monitor`, `unlock_monitor` e `kill_pid`. A mensagem recebida é interpretada para identificar a ação e seus parâmetros. Para encerrar um processo, o PID é convertido para o formato esperado. Nomes de comandos não registrados produzem erro, em vez de serem tratados como instruções genéricas do sistema operacional.

A execução depende do contexto da sessão e das permissões disponíveis. Um processo pode terminar antes de a ação chegar, ou a sessão gráfica pode não estar ativa. Por isso, a aceitação da solicitação pela API e a obtenção da ação pelo agente devem ser distinguidas do resultado final. O fluxo atual registra eventos localmente, mas não apresenta confirmação completa de execução retornada ao painel.

7.3.3 Serviço e sessão do usuário no Windows

No Windows, o agente principal executa como serviço, separado da sessão gráfica do usuário. O agente-session executa no contexto interativo, onde pode alcançar aplicações e elementos da área de trabalho. Named Pipes transportam comandos do processo principal para a sessão e relatórios de PIDs no sentido inverso. A Figura 5 apresenta essa divisão de responsabilidades.

Figura 5 - Arquitetura do serviço e da sessão do usuário no Windows



Fonte: acervo do projeto MonitorEdu (2026).

Os nomes **agente.exe** e **agente-session.exe** correspondem aos componentes Windows. A indicação **session 0** representa o contexto do serviço; **session 1** ilustra

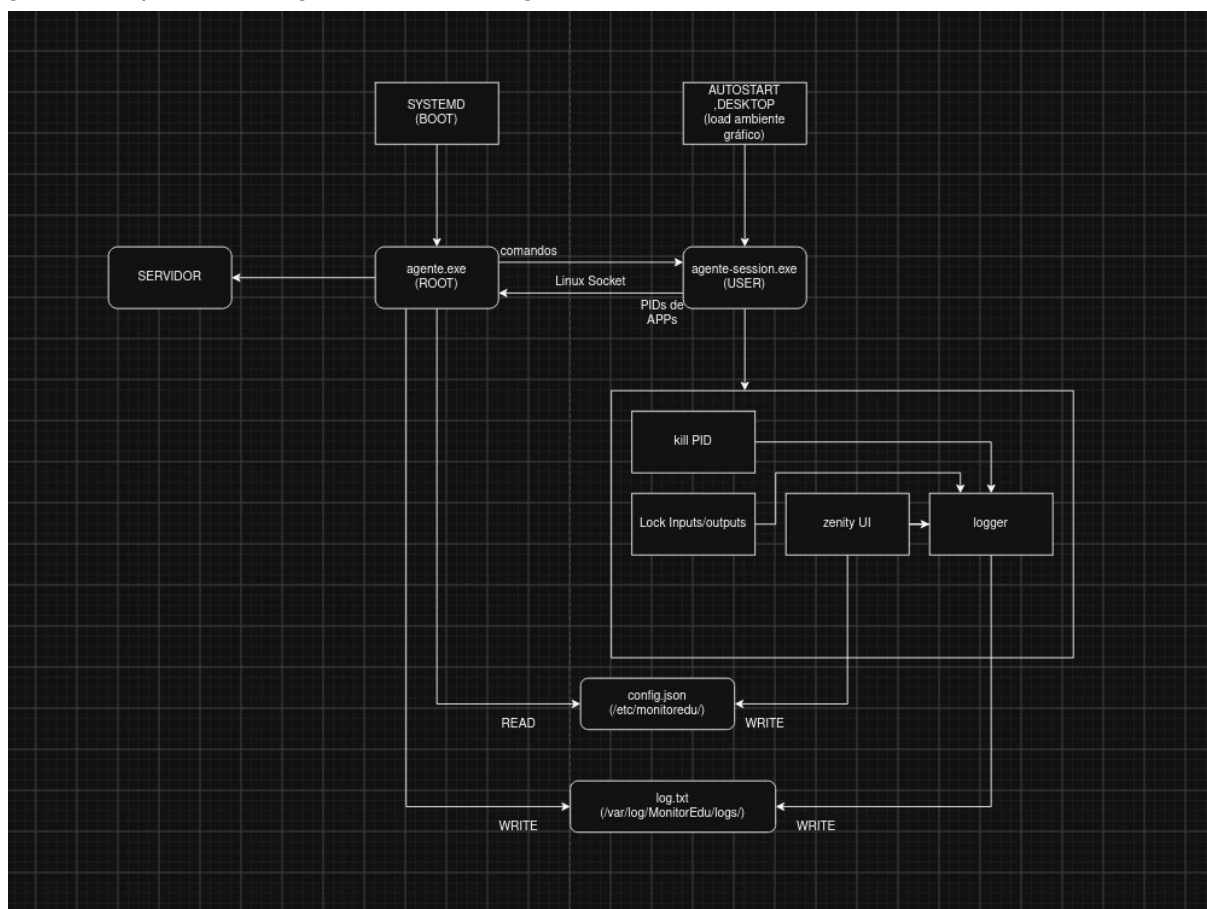
uma sessão interativa e não impõe que todo usuário receba sempre esse número. O diagrama destaca inicialização no logon por Task Scheduler. No instalador atual, a inicialização do componente de sessão é registrada pela chave Run do Registro do Windows; a separação funcional ilustrada permanece a mesma.

O arquivo config.json é mantido em C:\ProgramData\MonitorEdu. Os eventos são registrados no subdiretório logs, e o nome efetivamente utilizado no código é agent.log. O rótulo log.txt do diagrama é uma representação do arquivo de eventos. A descrição técnica utiliza os nomes atuais, preservando o conteúdo do material gráfico.

7.3.4 Serviço e sessão do usuário no Linux

No Linux, a unidade systemd inicia o processo principal, e um arquivo .desktop de autostart inicia o componente da sessão gráfica. A comunicação entre esses processos utiliza sockets Unix. O serviço recebe as métricas e os relatórios da sessão e encaminha as ações ao processo que possui o contexto gráfico necessário. A Figura 6 apresenta essa organização prevista para a distribuição Linux.

Figura 6 - Arquitetura do agente e da sessão gráfica no Linux



Fonte: acervo do projeto MonitorEdu (2026).

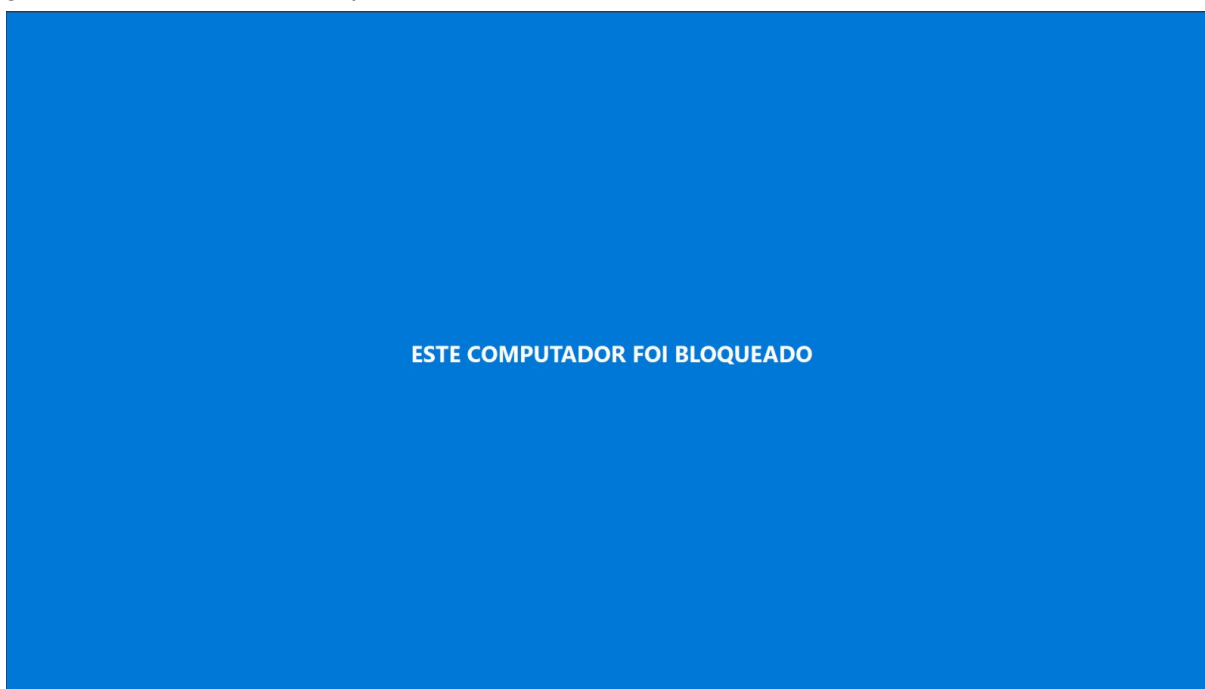
O diagrama conserva os rótulos `agente.exe` e `agente-session.exe`, mas os scripts Linux geram os binários `monitoredu` e `monitoredu-session`, sem extensão `.exe`, instalados em `/usr/bin`. A configuração utiliza `/etc/monitoredu/config.json`. O código de logs aponta para `/var/log/MonitorEdu/logs/agent.log`. Os rótulos da imagem são interpretados como nomes dos componentes, e não como comandos de instalação a reproduzir literalmente.

Os pacotes `.deb` atendem às distribuições baseadas em Debian, e os `.rpm` atendem a Fedora e sistemas compatíveis com esse formato. Ambos permanecem em fase de finalização. As rotinas de monitor e entrada utilizam ferramentas ligadas ao ambiente X11; dessa implementação não se conclui suporte completo a Wayland. A validação deve verificar sessão gráfica, dependências e permissões em cada ambiente utilizado.

7.3.5 Bloqueio e desbloqueio do monitor

No Windows, o bloqueio do monitor cria uma janela sobre a área de trabalho com a mensagem de bloqueio e aciona a rotina de bloqueio de teclado e mouse. O desbloqueio fecha essa janela e libera os dispositivos de entrada. A Figura 7 mostra o estado visual registrado durante o bloqueio. Trata-se de uma tela produzida localmente pelo agente, sem transmissão contínua da imagem ao servidor.

Figura 7 - Estado visual do bloqueio de monitor no Windows



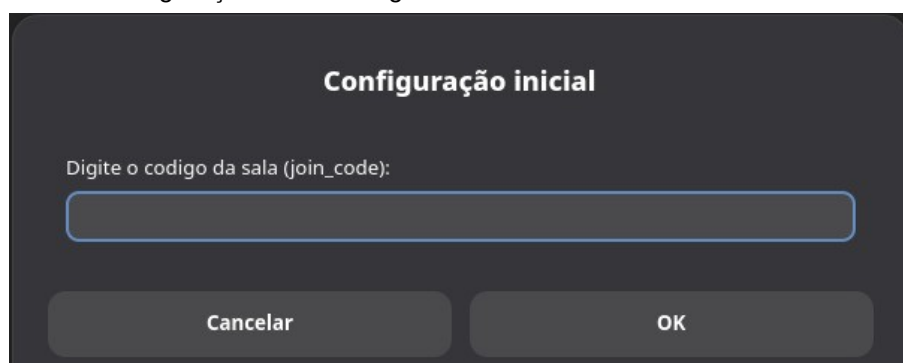
Fonte: acervo do projeto MonitorEdu (2026).

No Linux, a implementação consulta as saídas com xrandr e utiliza as opções de desligamento ou ativação dessas saídas. O controle de entrada utiliza xinput. Portanto, o efeito visual e o mecanismo técnico não são idênticos ao Windows. A reversão das ações e o comportamento após falhas precisam ser verificados nos ambientes da distribuição Linux, ainda em finalização.

7.3.6 Configuração inicial e registro da estação

O componente de configuração solicita o código da sala por uma janela gráfica. O programa remove espaços adicionais e recusa código vazio, gera o UUID e obtém o nome do computador. Em seguida, envia o registro à API. A configuração é salva após a aceitação do registro, permitindo que o processo principal reconheça a estação e inicie a operação. A Figura 8 apresenta a entrada do código de sala.

Figura 8 - Janela de configuração inicial do agente



Fonte: acervo do projeto MonitorEdu (2026).

Enquanto não encontra identificação e código de sala, o processo principal aguarda a configuração. O código de ingresso deve corresponder à sala criada pelo professor. O uso de uma janela reduz a necessidade de editar arquivos manualmente, mas o endereço da API e as demais definições continuam fazendo parte da configuração do agente. Erros de rede ou rejeição do registro devem ser tratados antes de considerar a instalação pronta para operar.

7.3.7 Distribuição para Windows e Linux

O instalador Windows é organizado em Inno Setup. Ele distribui os dois executáveis, registra o serviço, configura sua inicialização e inclui rotinas relacionadas ao DNS local. A instalação requer privilégios de administrador. Alguns identificadores internos e mensagens ainda conservam nomes anteriores do projeto; eles pertencem aos artefatos de distribuição e não alteram o nome MonitorEdu adotado na documentação.

Os scripts `build-deb.sh` e `build-rpm.sh` compilam o agente principal e o componente de sessão para Linux e organizam os arquivos dos pacotes. O `.deb` reúne metadados e scripts próprios desse formato; o `.rpm` utiliza a especificação do pacote e o `rpmbuild`. Os arquivos também incluem unidade `systemd`, autostart gráfico e configuração. A conclusão desse processo requer validar instalação, atualização, remoção e restauração das configurações afetadas em cada família de distribuições.

7.3.8 Registro e rotação de logs

O módulo logger registra eventos estruturados e erros das rotinas. A configuração utiliza rotação com tamanho máximo de 10 MB para o arquivo ativo, até

cinco arquivos de backup e retenção configurada de 30 dias, com compressão. O limite de 10 MB refere-se a cada arquivo ativo e não ao espaço total que os logs podem ocupar com os backups.

O espaço aproximado de 14,5 MB informado para a aplicação também deve ser separado do espaço ocupado pelos logs. Não é correto afirmar que a instalação ocupa sempre 24,5 MB pela soma automática desses valores. O arquivo ativo cresce conforme os eventos, e os backups dependem da rotação e da retenção. A avaliação de armazenamento deve observar a aplicação e os registros em momentos definidos.

7.4 Integração entre os componentes

A integração começa no cadastro do agente por código de sala. Depois, a estação envia métricas, a API atualiza o D1 e a interface consulta o estado disponível. Para uma ação administrativa, o professor seleciona a estação, a API registra o comando e o agente o recebe na resposta do envio de métricas. A comunicação local por IPC entrega a ação ao componente de sessão.

As políticas DNS utilizam sincronização própria: o painel define modo e listas, o Worker persiste os dados e o agente recebe a política durante sua consulta. Esse fluxo precisa ser observado junto à configuração do resolvedor local. A existência de uma política no banco não comprova, isoladamente, que a estação esteja utilizando o DNS do agente ou que a última sincronização tenha ocorrido sem erro.

7.5 Estado atual da implementação

A versão atual contém interface web, backend em Cloudflare Workers, persistência em D1 e agente em Go. Estão implementados o acompanhamento de métricas e processos, o conjunto de comandos locais, o painel administrativo e o bloqueio DNS. O Windows possui distribuição disponível, enquanto os pacotes Linux .deb e .rpm estão em finalização.

Os registros disponíveis ainda exigem validação de falhas, recuperação, autorização dos agentes e operação com várias estações. A retenção do estado recente e a ausência de confirmação de execução dos comandos delimitam a implementação atual.

8 TESTES E VALIDAÇÃO

8.1 Plano de testes

O plano considera o percurso completo das ações entre interface, API, banco e agente. Cada execução deve identificar versão utilizada, ambiente, configuração, procedimento e resultado observado. O estado registrado pela interface, a resposta da API e o efeito local precisam ser avaliados em conjunto, especialmente quando a solicitação depende de um ciclo posterior de polling.

A validação disponível reúne medidas do Positivo Master N1110, capturas de interfaces e análise das rotinas. Os registros não consolidam duração dos ensaios, repetições, tempos de resposta ou testes de carga; por isso, não permitem calcular médias, percentis ou taxas de sucesso.

8.2 Cenários e critérios de verificação

A Tabela 6 apresenta procedimentos e resultados esperados. Ela constitui o plano de verificação das funções, e não um relatório de aprovação de todos os casos. As evidências já disponíveis são discutidas separadamente.

Tabela 6 - Cenários de verificação funcional e de integração

Caso	Procedimento	Resultado esperado
CT01	Registrar estação com código de sala válido e repetir com código vazio ou inválido.	Estação válida associada à sala; entradas rejeitadas sem registro indevido.
CT02	Enviar métricas e consultar o painel após os ciclos de atualização.	Estado e identificação coerentes com os dados da estação.
CT03	Bloquear e desbloquear monitor, teclado e mouse em uma estação selecionada.	Ação aplicada ao alvo e reversão confirmada na sessão local.
CT04	Solicitar encerramento de processo de teste pelo PID e repetir com PID inexistente.	Processo-alvo encerrado quando possível; falha identificada sem atingir outro processo.
CT05	Aplicar blocklist e allowlist, verificando domínio, subdomínio e sincronização.	Consultas permitidas ou recusadas conforme a política recebida.
CT06	Acessar a administração com perfis distintos e cadastrar instituição e usuário autorizado.	Administração restrita e vínculo institucional salvo corretamente.
CT07	Interromper a comunicação com a API e restabelecê-la em seguida.	Falha identificada, comportamento local registrado e retomada observada.
CT08	Instalar, reiniciar e remover o agente nos ambientes Windows e Linux previstos.	Serviço e sessão iniciados conforme a distribuição; configurações afetadas verificadas.

Fonte: elaborado pelos autores (2026).

8.3 Testes de integração e tratamento de falhas

A integração deve verificar se o agente registrado pertence à sala correta e se o professor consegue consultar somente os recursos autorizados. Para comandos, o resultado esperado inclui o efeito observado na estação, pois o retorno de envio da interface não confirma a execução. A perda de uma resposta depois de o comando ser removido pelo servidor é um cenário relevante para avaliar a entrega atual.

Para DNS, os testes devem confirmar o uso de 127.0.0.1 pela estação, a atualização da política, a resolução dos domínios permitidos e a resposta de bloqueio para os recusados. A falha de sincronização precisa ser registrada separadamente da falha do resolvedor externo. O código conserva os domínios a reenviar quando a sincronização com a API falha; esse tratamento não substitui a observação de recuperação em um ensaio reproduzível.

8.4 Consumo observado do agente

No notebook de avaliação, foram informados consumo aproximado de 14 MB de memória RAM, uso de CPU inferior a 1% e ocupação aproximada de 14,5 MB pela aplicação. O arquivo de log possui limite de 10 MB por arquivo ativo. A Tabela 7 mantém separados os resultados observados e o parâmetro de configuração.

Tabela 7 - Consumo observado e limite de log do agente

Indicador	Valor informado	Interpretação
Memória RAM	Aproximadamente 14 MB	Consumo observado do agente no ambiente avaliado.
Uso de CPU	Inferior a 1%	Resultado informado para a observação inicial; não é um teto garantido para toda operação.
Aplicação em disco	Aproximadamente 14,5 MB	Ocupação informada para a aplicação, distinta dos logs.
Arquivo de log ativo	Limite de 10 MB	Parâmetro de rotação; não representa ocupação permanente nem limite total dos backups.

Fonte: registros de teste do projeto e configuração do módulo logger (2026).

Os resultados são vinculados ao Positivo Master N1110 com Intel Celeron N4020, 4 GB de RAM, 128 GB de armazenamento e Windows 11 Pro. A capacidade da máquina não deve ser apresentada como mínimo absoluto do agente. Da mesma forma, 14 MB observados não definem a quantidade total de memória necessária ao sistema operacional e às aplicações usadas em aula.

A rotina CollectSelf mede o processo em que o agente principal está executando. Para avaliar toda a instalação, um protocolo futuro deve identificar separadamente esse processo, o componente de sessão, a duração de observação e as ações executadas. Os dados disponíveis não permitem estimar o consumo total do navegador, do serviço em nuvem ou de todas as estações de um laboratório.

8.5 Evidências e alcance da validação

As capturas de configuração, administração e bloqueio registram estados concretos das interfaces. Os diagramas mostram a organização dos componentes, enquanto o código permite localizar a implementação das rotinas descritas. As medidas de consumo acrescentam uma observação quantitativa inicial. Esses materiais sustentam a documentação do protótipo, mas não constituem, em conjunto, um teste de carga ou uma avaliação com usuários.

A validação completa ainda requer registros de repetição, falhas de rede, reversão dos comandos, permissões e compatibilidade dos pacotes Linux. Não se atribuem ao sistema tempos de resposta, disponibilidade, quantidade máxima de estações ou taxa de sucesso que não tenham sido medidos. Essa delimitação permite utilizar os resultados existentes sem ampliar seu alcance experimental.

9 RESULTADOS E DISCUSSÃO

9.1 Resultados obtidos

O desenvolvimento resultou em uma solução com interface React, API em Cloudflare Workers, persistência em D1 e agente local em Go. O projeto reúne salas e estações, coleta métricas, apresenta processos e encaminha comandos administrativos definidos. O painel administrativo e a política de bloqueio DNS já fazem parte do estado atual, enquanto a distribuição Linux permanece em finalização.

A separação entre serviço e sessão do usuário permite organizar funções que dependem de contextos distintos no computador. Os materiais gráficos registram essa organização e interfaces da aplicação. As medições iniciais mostram baixo consumo do agente no equipamento avaliado, mas não demonstram isoladamente estabilidade sob carga, compatibilidade universal ou benefício pedagógico.

9.2 Relação com os objetivos definidos

A Tabela 8 relaciona os objetivos da Introdução à implementação e às evidências disponíveis. A situação parcial indica que existe desenvolvimento realizado, mas permanece uma etapa de distribuição ou validação necessária ao atendimento completo do objetivo.

Tabela 8 - Relação entre objetivos e resultados do projeto

Objetivo	Resultado e alcance
Centralizar o acompanhamento das estações	Interface e API organizadas por salas e agentes, com estado recente de métricas.
Disponibilizar comandos administrativos	Conjunto de ações locais implementado; confirmação completa de execução e cenários de falha ainda exigem validação.
Aplicar bloqueio por DNS	Políticas de bloqueio e permissão implementadas e funcionais; alcance condicionado ao uso do resolvidor local.
Oferecer configuração inicial acessível	Janela de ingresso por código de sala e registro com UUID gerado pelo agente.
Organizar módulos e facilitar manutenção	Responsabilidades separadas entre interface, Worker e módulos do agente.
Manter Windows e finalizar Linux	Windows disponível; pacotes .deb e .rpm em fase de finalização.
Avaliar funcionamento e consumo	Medições iniciais e registros visuais disponíveis; protocolo ampliado de integração e desempenho ainda necessário.

Fonte: elaborado pelos autores (2026).

9.3 Interpretação do comparativo de consumo

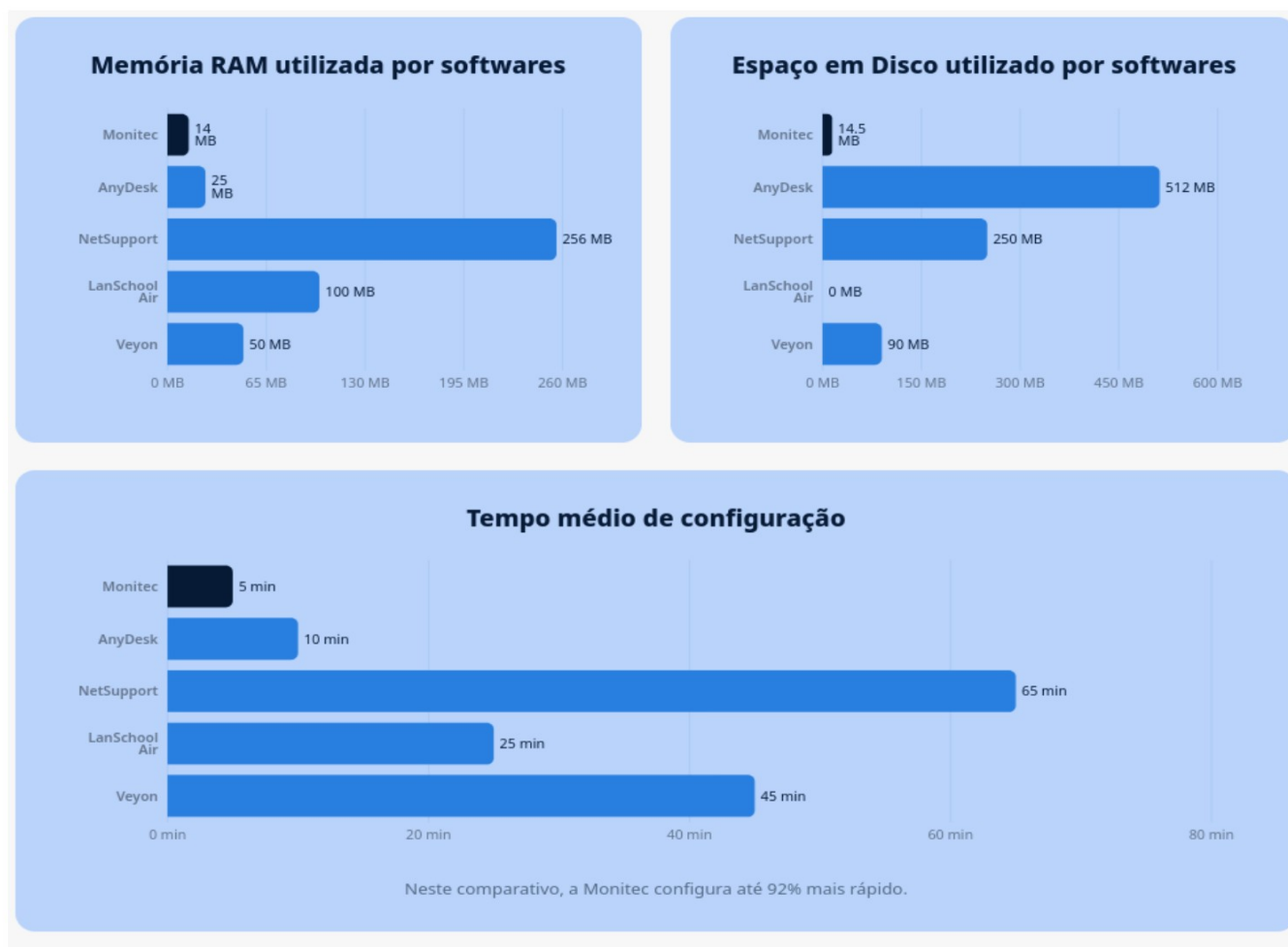


Figura 9 - Comparativo visual de consumo e requisitos dos softwares analisados

Fonte: acervo do projeto MonitorEdu (2026).



Positivo Master N1110
processador intel Celeron n4020
4GB RAM
Windows 11 PRO

A Figura 9 reúne medições do MonitorEdu e dados publicados pelos softwares de referência. As barras apresentam medidas de natureza diferente e não correspondem a um ensaio comparativo realizado no mesmo equipamento.

Os 90 MB associados ao Veyon correspondem ao consumo adicional do Veyon Master por computador monitorado, dependente da resolução. Esse valor não é uma medição do agente Veyon executado no mesmo notebook do MonitorEdu. Os valores de 250, 400 e 600 MB do NetSupport School indicam espaço livre exigido para modalidades de instalação, e não um benchmark de consumo durante as mesmas ações.

Para o MonitorEdu, aproximadamente 14,5 MB referem-se à aplicação, e o log pode crescer até o limite configurado por arquivo ativo, além dos backups da rotação. Assim, o gráfico não permite calcular economia percentual direta, afirmar superioridade geral ou determinar requisitos mínimos por comparação entre as barras.

9.4 Limitações identificadas

Os dados de desempenho estão restritos ao equipamento e à observação inicial informados. Não há resultados consolidados de duração, repetição, tamanho de laboratório ou carga concorrente. O envio periódico também implica atraso entre coleta, persistência e consulta do painel. Essas condições limitam afirmações sobre desempenho em tempo real e capacidade de atendimento.

A entrega de comandos remove a ação quando ela é devolvida ao agente, sem confirmação posterior da execução. As métricas conservam o estado recente, e a lista de processos possui seleção limitada. O filtro DNS atua sobre consultas que passam pelo resolvedor local. A implantação ampliada exige avaliar autenticação das estações, permissões de IPC, comportamento após falhas e restauração das configurações de rede.

A distribuição Linux ainda requer finalização e ensaios dos pacotes .deb e .rpm. Os recursos gráficos dependem do ambiente e das ferramentas utilizadas, sem comprovação de suporte completo a Wayland.

10 CONSIDERAÇÕES FINAIS

10.1 Síntese do trabalho

O MonitorEdu foi desenvolvido para centralizar informações de computadores de laboratórios escolares e permitir ações administrativas autorizadas. A arquitetura atual integra interface web, servidor em Cloudflare Workers, persistência em D1 e agente em Go. A organização por salas e estações aproxima os dados e os controles do contexto de utilização pelo professor.

O trabalho apresenta a implementação de métricas, processos, comandos locais, painel administrativo e políticas DNS. A configuração por código de sala e a separação entre serviço e sessão do usuário compõem a solução de implantação. A versão Windows está disponível, enquanto os pacotes Linux .deb e .rpm permanecem em fase de finalização.

As observações iniciais no Positivo Master N1110 indicam aproximadamente 14 MB de RAM, CPU inferior a 1% e aplicação com aproximadamente 14,5 MB em disco. Esses valores caracterizam o agente no contexto avaliado e não estabelecem requisitos mínimos absolutos nem resultados de um laboratório completo. A documentação registra também os limites de entrega de comandos, persistência do estado recente e alcance do filtro DNS.

O objetivo de construir uma base funcional para acompanhamento e administração das estações foi atendido no escopo implementado. A avaliação de compatibilidade, recuperação de falhas e operação em várias estações permanece necessária para ampliar a validação. Não se concluem ganhos de aprendizagem ou eficiência comparativa entre produtos sem estudos específicos para essas questões.

10.2 Trabalhos futuros

A continuidade imediata envolve finalizar os pacotes Linux e validar instalação, atualização e remoção nas famílias Debian e Fedora compatíveis. Também devem ser examinadas as dependências das ações gráficas e o comportamento em diferentes sessões, incluindo a necessidade de adaptações para Wayland.

Na comunicação, uma melhoria é registrar a confirmação de execução dos comandos e distinguir solicitação, entrega, execução e falha. A autenticação específica dos agentes, as permissões de IPC e a proteção da configuração local também devem ser revisadas. Para desempenho, recomenda-se um protocolo reproduzível com duração, repetições, ações executadas e quantidade de estações definidas, observando separadamente o serviço e o componente de sessão.

Uma etapa posterior pode avaliar usabilidade com professores em situações autorizadas de laboratório, registrando tarefas, dificuldades e resultados sem confundir uso técnico com impacto pedagógico. Recursos como histórico de métricas e relatórios podem ser estudados conforme a necessidade de armazenamento e o objetivo de acompanhamento. Painel administrativo e bloqueio DNS não são tratados como trabalhos futuros, pois já integram a versão atual.

REFERÊNCIAS

3DBMONITORAEDU. TCC MonitorEdu. Repositório de código-fonte. GitHub, 2026. Revisão eef68ef. Disponível em: <https://github.com/3DBMonitoraEdu/TCC>. Acesso em: 30 set. 2026.

ANYDESK. Install AnyDesk. 2025. Disponível em: <https://support.anydesk.com/docs/install-anydesk>. Acesso em: 30 set. 2026.

ANYDESK. Configure firewalls for AnyDesk. 2026a. Disponível em: <https://support.anydesk.com/docs/firewall>. Acesso em: 30 set. 2026.

ANYDESK. Supported operating systems. 2026b. Disponível em: <https://support.anydesk.com/docs/supported-operating-systems>. Acesso em: 30 set. 2026.

CLOUDFLARE. Cloudflare D1. Documentação técnica, 2026a. Disponível em: <https://developers.cloudflare.com/d1/>. Acesso em: 30 set. 2026.

CLOUDFLARE. Cloudflare Workers. Documentação técnica, 2026b. Disponível em: <https://developers.cloudflare.com/workers/>. Acesso em: 30 set. 2026.

EXPRESS. Express Node.js web application framework. 2026. Disponível em: <https://expressjs.com/>. Acesso em: 30 set. 2026.

FIELDING, R.; NOTTINGHAM, M.; RESCHKE, J. HTTP Semantics. RFC 9110. IETF, 2022. DOI: 10.17487/RFC9110. Disponível em: <https://www.rfc-editor.org/rfc/rfc9110.html>. Acesso em: 30 set. 2026.

GO. Documentation. 2026. Disponível em: <https://go.dev/doc/>. Acesso em: 30 set. 2026.

LANSCHOOL AIR. Installing LanSchool Air for Windows. 2026a. Disponível em: <https://helpdesk.lanschoolair.com/portal/en/kb/articles/installing-lanschool-air-for-windows-student>. Acesso em: 30 set. 2026.

LANSCHOOL AIR. LanSchool Air System Requirements. 2026b. Disponível em: <https://helpdesk.lanschoolair.com/portal/en/kb/articles/lanschool-air-system-requirements>. Acesso em: 30 set. 2026.

MOCKAPETRIS, P. Domain names concepts and facilities. RFC 1034. IETF, 1987. DOI: 10.17487/RFC1034. Disponível em: <https://www.rfc-editor.org/rfc/rfc1034.html>. Acesso em: 30 set. 2026.

NETSUPPORT SCHOOL. Pré-Requisitos do Sistema. 2026. Disponível em: https://help.netsupportschool.com/pt-br-windows/Content/Windows/Technical/system_requirements.html. Acesso em: 30 set. 2026.

NIELSEN, Jakob. 10 Usability Heuristics for User Interface Design. Nielsen Norman Group, 1994. Revisado em 30 jan. 2024. Disponível em: <https://www.nngroup.com/articles/ten-usability-heuristics/>. Acesso em: 30 set. 2026.

REACT. Quick Start. Documentação técnica, 2026. Disponível em: <https://react.dev/learn>. Acesso em: 30 set. 2026.

SQLITE. About SQLite. 2026. Disponível em: <https://www.sqlite.org/about.html>. Acesso em: 30 set. 2026.

VEYON. Installation. Veyon 4.10.4 Documentation, 2026. Disponível em: <https://docs.veyon.io/en/latest/admin/installation.html>. Acesso em: 30 set. 2026.